SCANIFY AI

Smart Document Scanner using Image Processing and Computer Vision

Subject	Image Processing and Computer Vision (IPCV)
Section / Team	Section C / Team 2
Submitted to	Abhimanyu Sahu Sir
Date	Friday, 30 October 2026

Team Members: Pragya Tiwari (20233211) • Muskan Dubey (20233193) • Munde Pratik Hanumant (20233192) • Puli Laxmi Vennela (20233224)

Abstract

Scanify AI is a document-scanning application that converts ordinary photographs of paper documents into cleaner and more useful digital documents. The project applies classical image-processing and computer-vision operations to detect the page boundary, correct perspective distortion, improve readability, and prepare the image for Optical Character Recognition (OCR). Its IPCV pipeline uses OpenCV, NumPy, Pillow, and Tesseract OCR through pytesseract. The application also supports crop, rotate, resize, compression, multiple scan modes, text extraction, PDF generation, and persistent document storage through Supabase. Extracted text can be reused for search, summary, and question-answering features. The project demonstrates how geometric correction, edge detection, contour analysis, thresholding, and OCR can be combined into a practical end-to-end document digitization system.

1. Introduction

Paper documents are still common in academic, administrative, and personal work, but a simple mobile photograph is rarely equivalent to a scanned page. Images may contain skew, background clutter, uneven illumination, shadows, low contrast, or rotation. These problems reduce readability and can significantly affect OCR performance. Scanify AI addresses this problem by improving the visual and geometric quality of the document before converting the processed image into a reusable digital form.

The main objective is to create a scanner-like workflow without requiring dedicated scanning hardware. A user can upload or capture a document image, automatically detect the page, obtain a top-down corrected view, apply enhancement modes, extract text, and export or save the result. The project combines core IPCV concepts with a practical document-management application.

2. Methodology

The workflow is organized as a sequence of transformations. Each stage has a specific purpose: normalize the input, identify document geometry, correct the view, enhance the page, and finally convert the visual content into text or a document file.

2.1 System Workflow

1 Input Image	2 Pre-processing	3 Document Detection	4 Perspective Correction	5 Enhancement + OCR	6 PDF / Saved Workspace

2.2 Image Acquisition and Pre-processing

The input can be a captured or uploaded document image. The image is decoded and its orientation is normalized using available metadata. Very large images can be resized or compressed so that processing remains efficient while preserving enough detail for text recognition. For automatic document detection, a working copy is converted to grayscale and smoothed to suppress small texture and noise.

2.3 Document Boundary Detection

Canny edge detection is applied to emphasize strong intensity transitions corresponding to page boundaries and printed structures. Morphological operations can connect weak or broken edge segments. Contours are then extracted and ranked by area. Polygon approximation is applied to larger candidates, and a four-point contour is treated as the most likely document boundary when it satisfies geometric checks. This method is computationally light, interpretable, and suitable for interactive document-scanning workflows.

2.4 Perspective Transformation

After obtaining the four document corners, the points are ordered as top-left, top-right, bottom-right, and bottom-left. The required rectangular output dimensions are estimated from opposite side lengths. OpenCV computes a perspective transformation matrix that maps the skewed quadrilateral in the original photograph to a rectangular plane. The resulting image resembles a top-down scanner view even when the original photograph was taken at an angle. If a reliable page boundary is not found, the system avoids applying an unsafe transformation and keeps the original geometry for manual correction.

2.5 Scan Enhancement Modes

The corrected page can be displayed in multiple modes such as Colour, Clean, Gray, and Black & White. These modes use operations including contrast adjustment, illumination correction, grayscale conversion, denoising, sharpening, and thresholding. Black & White is particularly useful for high-contrast text documents, while Colour is better for pages containing diagrams, stamps, or coloured annotations. Manual crop, rotate, resize, compression, and filter controls provide an additional correction path when automatic processing is insufficient.

2.6 OCR and Text Extraction

Optical Character Recognition is performed with Tesseract through the pytesseract Python interface. Before OCR, the processed image is converted to grayscale. Small text regions can be enlarged using bicubic interpolation, followed by denoising and Otsu thresholding. Otsu's method automatically selects a binary threshold from the image histogram, helping separate dark characters from the background. The cleaned image is then passed to Tesseract, which returns machine-readable text. This text can be copied, saved, downloaded, or used by later application features.

2.7 PDF Generation and Persistent Workspace

Processed scan images can be exported as PDF pages, and multiple processed pages can be combined into a document. Supabase is used for persistent application data so that user-specific information and saved documents can remain available after the current browser session. This turns the project into a reusable document workspace in which scanned content can be reopened and managed.

2.8 AI-Assisted Document Features

Once OCR has converted the image into text, the application can use the recognized content for higher-level operations such as short summaries, detailed summaries, study-oriented summaries, and document question-answering. These features operate on OCR-generated text after the scanning stage and remain separate from the core IPCV image-processing pipeline.

2.9 Core Algorithm

- Load the document image and normalize its orientation.
- Create a resized working copy and convert it to grayscale.
- Smooth the image and apply Canny edge detection.
- Use morphology, contour detection, and polygon approximation to find a four-corner page candidate.
- Order the corners and apply perspective transformation to obtain a top-down view.
- Apply the selected scan enhancement mode or manual editing operation.
- For OCR, preprocess with grayscale, optional upscaling, denoising, and Otsu thresholding.
- Run Tesseract OCR and return recognized text.
- Export the processed image as PDF and/or save the document in the user workspace.

2.10 Team Contributions

Responsibilities across implementation, testing, and presentation were divided as follows:

Team Member	Roll No.	Contribution / Presentation Area
Pragya Tiwari	20233211	Application integration, Supabase persistence, document workspace, and final report/presentation integration.
Muskan Dubey	20233193	Image preprocessing, scan enhancement modes, and manual image-editing operations.
Munde Pratik Hanumant	20233192	Document boundary detection, contour analysis, and perspective-correction workflow.
Puli Laxmi Vennela	20233224	OCR pipeline, PDF/output workflow, functional testing, and demonstration support.

3. Results and Discussion

The completed system was evaluated through practical document-scanning demonstrations rather than a large benchmark dataset. Representative document photographs were passed through the complete workflow, and the output was checked for geometric correction, readability, text extraction, and downstream document operations. The implementation successfully demonstrates the expected behaviour of an IPCV document scanner under normal conditions.

Stage	Observed Result	Discussion
Document detection	Page boundary identified from visible edges.	Works best when the sheet contrasts with the background and all major corners are visible.
Perspective correction	Skewed page converted to a top-down rectangular view.	Improves readability and produces a scanner-like appearance.
Enhancement modes	Colour, grayscale, and binary-style outputs available.	Different modes are useful for text pages versus diagrams or coloured content.
Manual editing	Crop, rotate, resize, compression, and filters available.	Provides recovery when automatic detection is imperfect.
OCR	Processed image converted to machine-readable text.	Recognition quality depends on focus, font size, illumination, and scan cleanliness.
PDF / persistence	Processed scans can be exported and saved for later use.	Extends the project from image processing to a reusable document workflow.

3.1 Discussion of IPCV Performance

The strongest part of the system is the combination of contour-based page detection and perspective transformation. A photographed document is commonly represented as a quadrilateral in the image plane; detecting a four-corner contour therefore provides a direct route to geometric correction. This classical method does not require training data and is fast enough for interactive use. Its main limitation is that weak page boundaries, highly cluttered backgrounds, curved pages, or severe shadows can make the four-corner contour ambiguous.

Image enhancement also has a direct impact on OCR. Grayscale conversion reduces colour complexity, denoising removes small variations, and thresholding improves separation between foreground text and background. However, excessive thresholding can remove faint strokes, while preserving colour may be necessary for diagrams or highlighted notes. Providing multiple scan modes is therefore more useful than forcing one fixed enhancement pipeline for every document.

The OCR output is suitable for demonstrating extraction of printed text, but no formal Character Error Rate (CER) or Word Error Rate (WER) benchmark was performed. The current results should therefore be interpreted as functional implementation results rather than a statistical accuracy study. A quantitative extension would require a labeled test set and comparison of recognized text with ground truth.

3.2 Limitations Observed

- Low contrast between paper and background can reduce boundary-detection reliability.
- Curved or folded pages do not perfectly satisfy the planar perspective model.
- Blur, tiny fonts, handwriting, and uneven lighting can reduce OCR quality.
- Tesseract language support depends on the language packs installed on the machine.
- Errors in OCR text can propagate to summary and question-answering features.

4. Conclusion

Scanify AI demonstrates a complete application of Image Processing and Computer Vision to document digitization. The system accepts a photographed or uploaded page, detects its boundary, corrects perspective distortion, improves visual quality, and converts the processed page into machine-readable text using OCR. The important IPCV concepts demonstrated in the project include grayscale conversion, smoothing, Canny edge detection, morphology, contour extraction, polygon approximation, perspective transformation, denoising, and Otsu thresholding. Together, these operations transform an ordinary mobile photograph into a much more scanner-like document.

The project also shows how IPCV can be integrated with practical software features. PDF generation makes the output shareable, Supabase provides persistent user-oriented storage, and OCR text can support search, summary, and question-answering functions. The system therefore goes beyond a single image-processing algorithm and implements an end-to-end workflow from raw image acquisition to reusable digital information.

The implementation exposes each major processing stage directly - input image, automatic page detection, perspective-corrected scan, enhancement mode, OCR output, and final PDF/saved document - making the workflow easy to demonstrate and evaluate. Future improvements can include stronger handling of curved pages, automatic blur detection, multilingual OCR, searchable PDFs, OCR confidence highlighting, and quantitative evaluation using CER and WER.

Practical Demonstration Sequence

- Open the application and demonstrate user login/workspace access.
- Upload or capture a document photograph.
- Show automatic page detection and perspective correction.
- Switch between Colour, Clean, Gray, and Black & White modes.
- Demonstrate at least one manual operation such as crop, rotate, or resize.
- Run OCR and display the extracted text.
- Generate/export a PDF and save or reopen the document from the workspace.
- Briefly demonstrate summary or document Q&A using the OCR text.

References

1. OpenCV Documentation - image processing, Canny edge detection, contours, and perspective transforms.

2. Tesseract OCR Documentation - optical character recognition engine.

3. pytesseract Documentation - Python interface to Tesseract.

4. Pillow (PIL) Documentation - image loading, resizing, orientation handling, and export.

5. Supabase Documentation - authentication, database, and persistent application data.